CHAPTER 1

What “learning” means

When I started learning AI as an engineering student, I kept hearing new words: machine learning, deep learning, LLM, RAG, agents. I thought I had to learn seven different subjects. Later I understood that all these words belong to one story. Each new idea came because the old idea had a problem it could not solve.

In this chapter I will tell that story from the beginning, in simple steps. At the end, you will see where this book fits.

BY THE END OF THIS CHAPTER

- Understand the difference between a computer that follows rules and a computer that learns.
- Explain what a *model* is, in one sentence.
- See the full picture: AI, machine learning, deep learning, LLMs, RAG, agents and Jev.

Rules work, until they don’t

Many systems in our daily life follow simple rules. A water tank switch turns the motor off when the tank is full. Your bank blocks a UPI payment if it is above your daily limit. A college portal shows “absent” if attendance is below 75%. Nobody “trained” these systems. A programmer wrote the rule, and the computer follows it.

Rules are good. They are fast and cheap, and when something goes wrong you can read the rule and understand why. If a simple rule solves your problem, use the rule.

The problem starts when the situation keeps changing. Think about fake messages that try to steal passwords. This is called **phishing**. The IT team of a company writes a rule: *if an email has the word “password”, mark it as suspicious.* On Monday, the rule blocks the IT team’s own “reset your password” emails, so they add an exception. On Wednesday, the attacker writes “password” with a zero, so they add another rule. On Friday, the

attacker stops using the word completely and just sends a link to a fake login page. The list of rules keeps growing, and it is always one step behind the attacker. You may have seen the same thing with fake “KYC update” SMS messages: the wording changes every week.

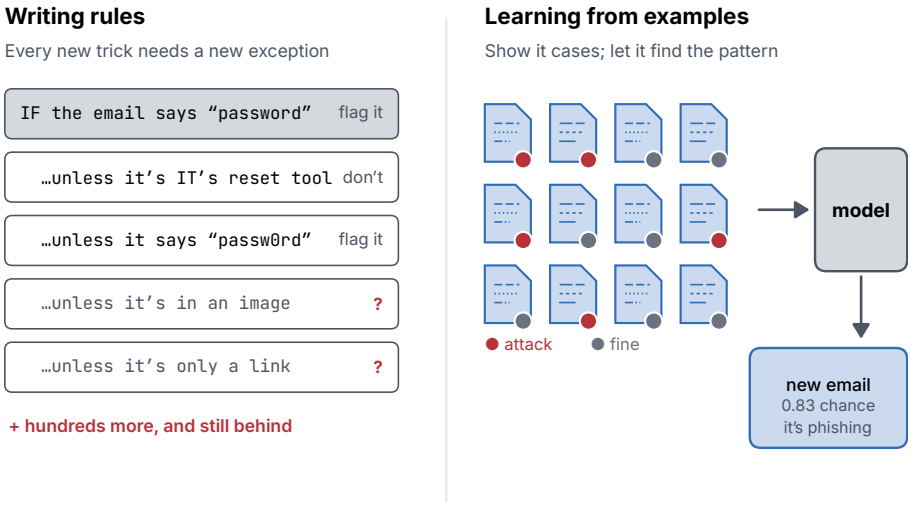


Figure 1.1. Two ways to catch phishing. On the left, every new trick forces a new exception, and the list never catches up. On the right, we stop writing rules and show the computer labelled examples instead.

So we change the approach. Instead of writing the rules ourselves, we collect thousands of old emails where we already know the answer: this one was phishing, this one was normal. Then we ask the computer to find the pattern by itself (Figure 1.1).

● KEY IDEA

Machine learning means the computer finds the rules by itself, by studying examples where we already know the correct answer.

What is a model?

After the computer studies the examples, we get something we can use on new cases. You give it a new email, and it gives back an answer. This thing is called a **model**.

You can imagine a model as a machine with many small knobs. During learning, the computer keeps turning the knobs until the machine gives the right answer for most of the examples. After that, we stop turning the

knobs and use the machine on new emails, where nobody knows the answer yet.

> ● KEY IDEA
>
> **A model is a function whose settings come from examples, not from a programmer.**

You already use many models. When your bank sends an SMS like *"Did you just make a payment of ₹40,000 in another city? Reply NO if not you,"* no person wrote a rule about you. A model learned your normal spending pattern, and this payment did not match it.

A model can learn the wrong thing

To make the ideas concrete, this book uses one example again and again: a logistics company I have invented, called Kestrel Logistics. Like any company, its computers raise warnings when something looks suspicious, for example a login from a strange country. These warnings are called **alerts**. In four weeks, the security team receives 20,000 alerts, about 714 per day. Only 7.6% of them are real attacks; the rest are false alarms. Because I generated these alerts with a program, I know the correct answer for every alert, so we can check every model properly.

Each alert comes with a *threat score* between 0 and 1. A higher score means outside security sources think it is more dangerous. Let us try a simple rule: *if the score is above some number t, mark the alert as an attack.* A cut-off number like t is called a **threshold**.

Which value of t should we choose? We let the computer try every value from 0 to 1, count the mistakes for each value, and keep the value with the fewest mistakes. This is learning in its smallest form.

The computer chooses 0.90. With this threshold it makes 1,487 mistakes. But look at this: a rule that *never* marks anything as an attack makes 1,515 mistakes, almost the same number. Out of 1,515 real attacks, the "best" rule catches only 73 (Figure 1.2).

Why did this happen? There are about twelve harmless alerts for every real attack. So the easiest way to reduce the number of mistakes is to almost never raise an alarm. The computer did exactly what we asked. Our question was wrong.

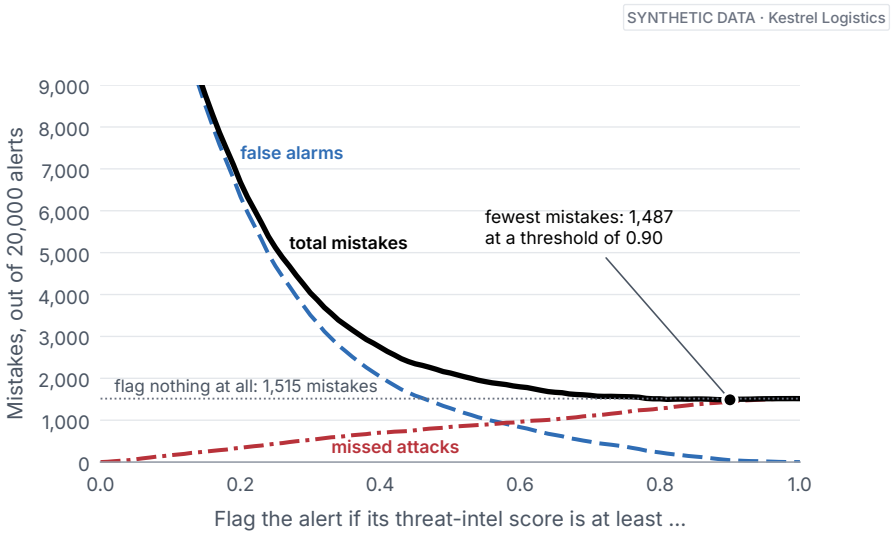


Figure 1.2. Every possible threshold and the mistakes it makes. Low thresholds bury the team in false alarms; high ones miss attacks. The total is lowest at the far right, just below the dotted line for “flag nothing at all”.

● KEY IDEA

A model learns only what we ask it to learn. If we ask the wrong question, it learns the wrong thing.

We made two mistakes in the question. First, we counted every mistake as equal. But missing a real attack is much more costly than one false alarm. Chapter 4 fixes this. Second, we forced the answer to be only *yes* or *no*.

A better answer: how likely?

Let us ask a different question: *among alerts with a similar score, how many were real attacks?*

Figure 1.3 shows the answer. There is no clear line where alerts suddenly become dangerous. Even in the lowest-score group, 2% were attacks. That looks small, but the group has 7,270 alerts, so more than a hundred attacks are hiding there. And in the highest-score group, only 62% were attacks.

So for an alert with a score of 0.55, the honest answer is not “attack” or “safe”. The honest answer is: *about one in five alerts like this is an attack.*

This kind of answer is much more useful. It tells the team how worried to be, which alerts to check first, and, once we know the cost of each mistake, where to draw the line.

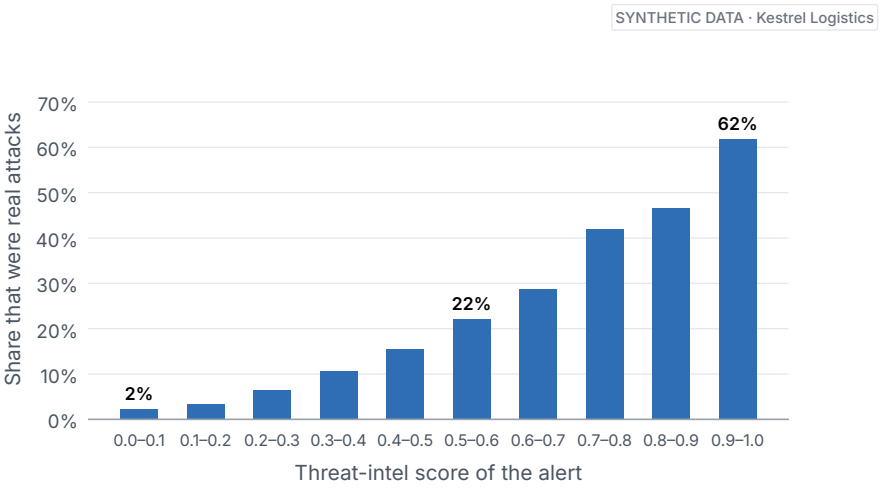


Figure 1.3. Group the alerts by threat score and count how many were real attacks. The share climbs steadily from 2% to 62%. Even at the top, plenty are harmless; even at the bottom, a few are attacks.

● KEY IDEA

A good model gives an answer and also says how sure it is. And that "how sure" must be correct.

When a model's "how sure" is correct, we say the model is **calibrated**. You will see this word many times in this book, and Chapter 3 explains it fully.

The bigger story, and where this book fits

The threat score was one simple number. But a photo has lakhs of pixels, and an email is a long sentence. Nobody can write rules for them by hand. So researchers built models with many layers that learn their own clues. This is **deep learning**. Then they trained very large deep-learning models on huge amounts of text, with one task: guess the next word. These models learned language itself. They are called **large language models**, or LLMs, like ChatGPT. An LLM knows a lot about the world but nothing about your company, so we give it your documents to read before it answers. This is called **RAG**. Finally, we let the model use tools and take actions step by step. This is an **agent**. Chapters 5 and 6 explain all of these.

When you watch an agent work, you notice something interesting. Most of its steps are not writing. They are small decisions with only a few possible answers: *Is this alert real? Which team should handle this ticket? Is this action*

safe? Today, people usually send each such question to an LLM. The LLM writes a full paragraph, and then more code has to find the actual answer inside that paragraph. It works, but it is slow and costly for such a small job.

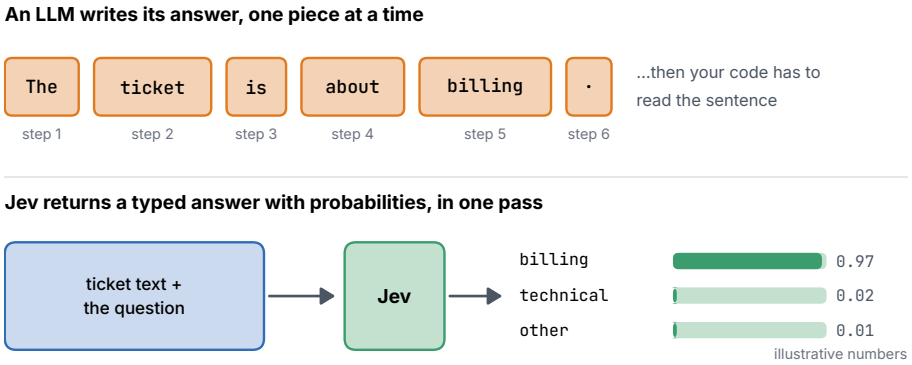


Figure 1.4. The same question, two ways. An LLM writes its answer piece by piece, and your code has to read the sentence. A decision model returns the answer as data, with a probability for every option, in one pass.

In 2026, a company called TypeSafe AI released a model named **Jev** for exactly these small decisions. Jev does not write any text. You give it the situation and a clear question, and it gives back the answer with a probability for each option (Figure 1.4). TypeSafe calls it a *System One* model. The name comes from the psychologist Daniel Kahneman, who called our fast, automatic thinking “System 1” (Kahneman 2011). I did not have access to Jev while writing this book, so every Jev answer here comes from a stand-in model I built that copies Jev’s public interface. Such numbers are marked *synthetic*.

Figure 1.5 shows the whole story on one page. The next three chapters stay at the base of this map, because everything above it depends on one idea: an answer should come with a probability, and that probability should be correct.

KEEP THESE

- **Machine learning** means the computer finds the rules by itself, by studying examples where we already know the correct answer.
- A **model** is a function whose settings come from examples, not from a programmer.
- A model learns only what we ask it to learn. If we ask the wrong question, it learns the wrong thing.
- A good model gives an answer and also says how sure it is. And that “how sure” must be correct.

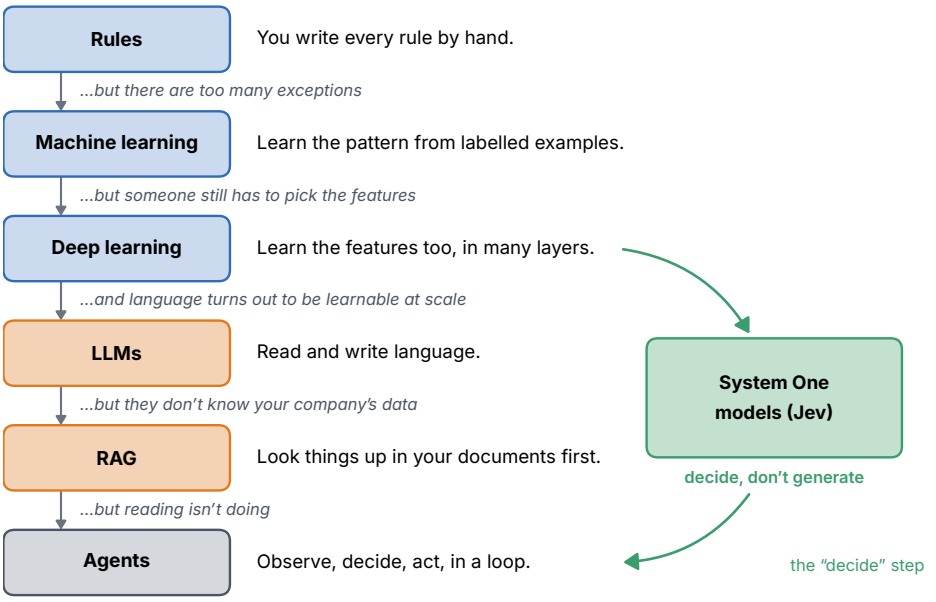


Figure 1.5. The story so far. Each step exists because the one before it hit a problem it couldn't solve. System One models like Jev grow out of deep learning and fit into the "decide" step of an agent.

EXERCISES

1. Name one system you use that follows fixed rules, and one that has learned from examples. How did you know?

Next chapter: what does "70% chance" really mean? There is a common trap here that even doctors fall into.